

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

...with a Leakage-Aware Evaluation Pipeline

Thai Nguyen Vu¹ Dung Van Bui² Nhut Tri Do² Van Du Nguyen³

FAIR'2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML
...with a Leakage-Aware Evaluation Pipeline

Thai Nguyen Vu¹ Dung Van Bui² Nhut Tri Do² Van Du Nguyen³
FAIR'2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

Seven minutes on a single claim: on CICIDS2017, a near-perfect F1 is usually a property of the evaluation pipeline, not of the model.

2026-09-26

Dimensionality Reduction and Feature Selection for Network
Intrusion Detection on Apache Spark ML

└ I. PROBLEM AND CONTRIBUTIONS

I. PROBLEM AND
CONTRIBUTIONS

I. PROBLEM AND CONTRIBUTIONS

A near-perfect F1 on CICIDS2017 is a warning sign

Two problems, usually treated separately

- **Cost.** ≈ 80 features per flow.
- **Trust.** Near-perfect binary F1 usually signals **data leakage**.

Two concrete leakage sources

1. `destination_port` encodes the *attacked service*.
2. **Feature-level duplicates** survive full-row dedup — the same sample lands in train *and* test.

This paper

Compare four reduction strategies over the same eight Spark ML classifiers — but first make the *evaluation pipeline* trustworthy.

The contribution is the protocol

Not “which method wins” but **how to measure so the answer means something**.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

I. PROBLEM AND CONTRIBUTIONS

A near-perfect F1 on CICIDS2017 is a warning sign

Frame the talk as a measurement paper. The port gives the model a port-to-label shortcut; 80 features are expensive to train, to score and to fit on an edge device. Both leakage sources are removed before any split.

A near-perfect F1 on CICIDS2017 is a warning sign

Two problems, usually treated separately

- **Cost.** ≈ 80 features per flow.
- **Trust.** Near-perfect binary F1 usually signals **data leakage**.

Two concrete leakage sources

1. `destination_port` encodes the attacked service.
2. **Feature-level duplicates** survive full-row dedup — the same sample lands in train and test.

This paper

Compare four reduction strategies over the same eight Spark ML classifiers — but first make the evaluation pipeline trustworthy.

The contribution is the protocol

Not “which method wins” but **how to measure so the answer means something**.

1. A **uniform comparison** of RF Importance, SHAP and PCA across the same 8 Spark ML algorithms + 2 ensembles on CICIDS2017.
2. SHAP contrasted **directly** against RF Importance on the same training set.
3. A **leakage-aware, statistically tested** pipeline: leak removal, deterministic feature-level dedup, validation-based selection, exact paired permutation test with Nadeau–Bengio correction.
4. A **per-attack-type multiclass** evaluation — where the methods genuinely differ.
5. An **edge-aware** recommendation: F1 *and* feature count *and* model size, for a Jetson Orin Nano Super (deployment reported in a companion systems paper).

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

I. PROBLEM AND CONTRIBUTIONS

Contributions

Five contributions; three, four and five are the ones rarely done together.

Contributions
1. A uniform comparison of RF Importance, SHAP and PCA across the same 8 Spark ML algorithms + 2 ensembles on CICIDS2017.
2. SHAP contrasted directly against RF Importance on the same training set.
3. A leakage-aware, statistically tested pipeline: leak removal, deterministic feature-level dedup, validation-based selection, exact paired permutation test with Nadeau–Bengio correction.
4. A per-attack-type multiclass evaluation — where the methods genuinely differ.
5. An edge-aware recommendation: F1 and feature count and model size, for a Jetson Orin Nano Super (deployment reported in a companion systems paper).

2026-09-26

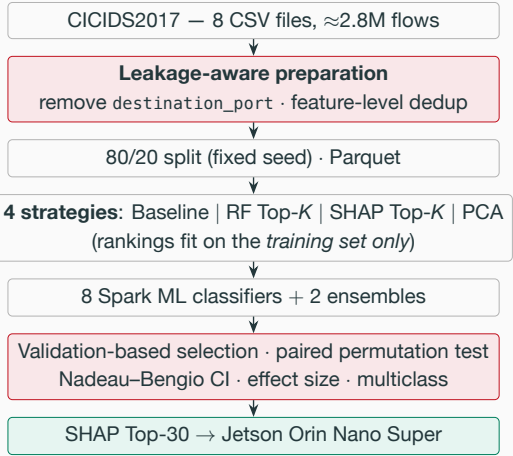
Dimensionality Reduction and Feature Selection for Network
Intrusion Detection on Apache Spark ML

└ II. LEAKAGE-AWARE PIPELINE

II. LEAKAGE-AWARE PIPELINE

II. LEAKAGE-AWARE PIPELINE

The leakage-aware pipeline



Removals happen before the split

- Feature-identical groups with **conflicting labels** are dropped *entirely*: **270 groups, 44,260 rows**.
- Result: 77 features, 1.79M / 0.45M train/test flows.

Design fixed a priori

- Exploration (K sweep) separated from confirmation.
- Hyper-parameters shared across all methods.
- The test set enters **no** selection step.

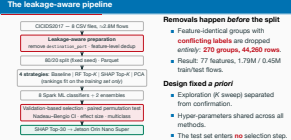
2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

II. LEAKAGE-AWARE PIPELINE

The leakage-aware pipeline

Deterministic deduplication is the part people get wrong. Conflicting groups are removed wholesale, so the result does not depend on how Spark partitions the data.



III. RESULTS AND CONCLUSION

Result 1: 60% of the features are removable

Method	Model	F1	Train (s)	Feat.
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0.9970	1226.1	30
PCA k=40	XGBoost	0.9939	1444.8	40
RF Top-30	XGBoost	0.9922	1295.1	30

Algorithm	RF Top-30	SHAP Top-30	Δ
Random Forest	0.9879	0.9955	+0.77%
XGBoost	0.9922	0.9970	+0.48%
GBT	0.9844	0.9918	+0.75%
Decision Tree	0.9835	0.9948	+1.15%

- SHAP Top-30 keeps baseline F1 while cutting **≈60% of features** and **≈73% of training time**.
- At *equal* feature count, SHAP beats RF Importance on every tree-based algorithm.
- PCA uses *k*=40 because it *extracts* rather than *selects* — each component carries only part of the variance. Cheaper than the baseline, but not explainable.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 1: 60% of the features are removable

Two tables, one message: SHAP Top-30 is the operating point. Same accuracy, 60 percent fewer features, and it is explainable, which matters for the SOC analyst and for the edge board downstream.

Result 1: 60% of the features are removable

Method	Model	F1	Train (s)	#feat.
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0.9970	1226.1	30
PCA k=40	XGBoost	0.9939	1444.8	40
RF Top-30	XGBoost	0.9922	1295.1	30

Algorithm	RF Top-30	SHAP Top-30	Δ
Random Forest	0.9879	0.9955	+0.77%
XGBoost	0.9922	0.9970	+0.48%
GBT	0.9844	0.9918	+0.75%
Decision Tree	0.9835	0.9948	+1.15%

- SHAP Top-30 keeps baseline F1 while cutting **≈60% of features** and **≈73% of training time**.
- At equal feature count, SHAP beats RF Importance on every tree-based algorithm.
- PCA uses *k*=40 because it *extracts* rather than *selects* — each component carries only part of the variance. Cheaper than the baseline, but not explainable.

Result 2: testing the difference instead of asserting it

Method	Model	F1 (mean ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009
paired permutation $p = 0.931$, $d = 1.94$		

Port ablation (RF)	F1	Recall (Attack)
Keep port	0.9965	0.9983
Remove port	0.9955	0.9961

Significant *only at the resolution floor*, gap ≈ 0.0001 : **practically equivalent**, so explainability and cost decide. After dedup the port’s isolated effect is small — duplicates were the dominant leak.

- $N=6$ paired *resamplings*: variance reflects data sampling, not just seeds.
- Two-sided permutation test, **exact enumeration** of all 2^6 sign flips: smallest reachable p is $2/2^6$, so $N \geq 6$ is *required*.
- Overlapping resamplings break t -independence → we use the **Nadeau–Bengio corrected CI**.

Nadeau & Bengio, *Inference for the Generalization Error*, Machine Learning 52(3), 2003.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 2: testing the difference instead of asserting it

Variance reflects data sampling, not just model initialization. The test is two-sided, and overlapping resamplings break t -independence. Small p , tiny effect in absolute terms: say plainly that we treat them as equivalent rather than claiming a winner.

Result 2: testing the difference instead of asserting it

Method	Model	F1 (mean ± CI95)
Baseline	XGBoost	0.99734 ± 0.00007
SHAP Top-30	XGBoost	0.99720 ± 0.00009
paired permutation $p = 0.931$, $d = 1.94$		

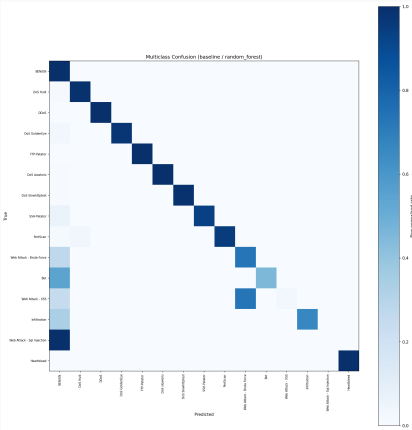
Port ablation (RF)	F1	Recall (Attack)
Keep port	0.9965	0.9983
Remove port	0.9955	0.9961

Significant only at the resolution floor, gap ≈ 0.0001 : **practically equivalent**, so explainability and cost decide. After dedup the port’s isolated effect is small — duplicates were the dominant leak.

Nadeau & Bengio, *Inference for the Generalization Error*, Machine Learning 52(3), 2003.

- $N=6$ paired resamplings: variance reflects data sampling, not just seeds.
- Two-sided permutation test, **exact enumeration** of all 2^6 sign flips: smallest reachable p is $2/2^6$, so $N \geq 6$ is required.
- Overlapping resamplings break t -independence → we use the **Nadeau–Bengio corrected CI**.

Result 3: binary F1 hides the rare classes



Class	Support	Recall	F1
Benign	380,119	0.9997	0.9989
DoS Hulk	34,446	0.9904	0.9947
Web – Brute Force	311	0.7299	0.7323
Bot	290	0.4552	0.6256
Web – XSS	111	0.0270	0.0526
Web – SQL Injection	6	0.0000	0.0000
weighted-F1 = 0.998		macro-F1 = 0.806	

- Two failure modes, read by destination:
- **Mislabeled but caught:** 81/111 XSS land on Web Brute Force — **76% still flagged** at the binary level.
 - **Leaking into Benign** (dangerous): all 6 SQL Injection, **54% of Bot**, 27% of Brute Force.

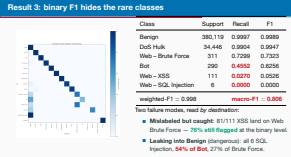
2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 3: binary F1 hides the rare classes

Weighted F1 0.998 versus macro F1 0.806. Three causes: extreme imbalance; removing the port hits web attacks hardest, because without the port-80 shortcut short HTTP flows look like browsing, so the 0.027 recall reported here replaces a leakage-inflated one; and Bot C&C beaconing resembles background traffic.



Result 4: the scores do not survive a different testbed

Train	Test	F1	AUC-PR
2017	→ 2017	0.9954	0.9997
2017	→ 2018	0.2154	0.4989
2018	→ 2018	0.9243	0.9675
2018	→ 2017	0.0536	0.1862

Mean of 4 runs, shared leakage-free features,
~2.39M flows

- In-domain stable *both* ways — cross-dataset AUC-PR **1.00→0.50** and **0.97→0.19**.
- Recall 0.126 / 0.030 at precision 0.850 / 0.248: the model **fails to recognize** the other testbed’s attacks.
- Cross-dataset F1 **not reproducible** (0.045–0.30) — report means over runs, judge on AUC-PR.
- Causes: CICFlowMeter version, network configuration, attack tooling.

Near-perfect in-domain scores — *even leakage-free ones* — do not certify deployability under distribution shift.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 4: the scores do not survive a different testbed

Do not soften this. In-domain 0.99, cross-dataset AUC-PR 0.50 / 0.19. If asked about the wide F1 interval: that is the finding — at this recall the metric is threshold-fragile, which is why we report means over runs and read the gap off AUC-PR. Consistent with prior cross-dataset IDS studies, and the reason domain adaptation is a prerequisite rather than a nice-to-have.

Result 4: the scores do not survive a different testbed

Train → Test	F1	AUC-PR
2017 → 2017	0.9954	0.9997
2017 → 2018	0.2154	0.4989
2018 → 2018	0.9243	0.9675
2018 → 2017	0.0536	0.1862

Mean of 4 runs, shared leakage-free features, ~2.39M flows.

- In-domain stable both ways — cross-dataset AUC-PR **1.00→0.50** and **0.97→0.19**.
- Recall 0.126 / 0.030 at precision 0.850 / 0.248: the model **fails to recognize** the other testbed’s attacks.
- Cross-dataset F1 **not reproducible** (0.045–0.30) — report means over runs, judge on AUC-PR.
- Causes: CICFlowMeter version, network configuration, attack tooling.

Near-perfect in-domain scores — even leakage-free ones — do not certify deployability under distribution shift.

Conclusion

What we showed

- SHAP Top-30 retains baseline F1 with **≈60% fewer features**.
- Baseline vs. SHAP Top-30 are **practically equivalent**.
- **macro-F1 0.806** against weighted-F1 0.998.
- Cross-dataset AUC-PR **collapses to 0.50 / 0.19**.

The contribution

- **How to evaluate reliably**, as much as which method wins.
- ## Next
- Full CSE-CIC-IDS2018; newer sets (RoEduNet, CIC-IoT).
 - Does the SHAP Top-30 *set itself* transfer?
 - Targeted imbalance handling; temporal features for beaconing.

Thank you — questions?

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Conclusion

Close on the protocol, not the leaderboard.

Conclusion

What we showed

- SHAP Top-30 retains baseline F1 with **≈60% fewer features**.
- Baseline vs. SHAP Top-30 are **practically equivalent**.
- **macro-F1 0.806** against weighted-F1 0.998.
- Cross-dataset AUC-PR **collapses to 0.50 / 0.19**.

The contribution

- **How to evaluate reliably**, as much as which method wins.

Next

- Full CSE-CIC-IDS2018; newer sets (RoEduNet, CIC-IoT).
- Does the SHAP Top-30 *set itself* transfer?
- Targeted imbalance handling; temporal features for beaconing.

Thank you — questions?

Key references

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Lanvin, M. et al. *Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes*. CRiSIS 2022. **label conflicts**

[3] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage pitfalls**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP**

[5] Nadeau, C. & Bengio, Y. *Inference for the Generalization Error*. Machine Learning 52(3), 2003. **corrected CI**

[6] D'hooge, L. et al. *Inter-dataset generalization strength of supervised machine learning methods for intrusion detection*. J. Information Security and Applications 54, 2020. **cross-dataset**

[7] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark ML**

[8] CIC & CSE. *CSE-CIC-IDS2018 dataset*. 2018. **cross-dataset target**

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Key references

Not presented — shown while taking questions, so sources are on screen if the audience asks. Full list is in the paper.

Key references	
[1]	Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. <i>Toward generating a new intrusion detection dataset and intrusion traffic characterization</i> . ICISSP 2018. CICIDS2017
[2]	Lanvin, M. et al. <i>Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes</i> . CRiSIS 2022. label conflicts
[3]	Arp, D. et al. <i>Dos and Don'ts of Machine Learning in Computer Security</i> . USENIX Security 2022. leakage pitfalls
[4]	Lundberg, S. M. & Lee, S.-I. <i>A unified approach to interpreting model predictions</i> . NeurIPS 2017. SHAP
[5]	Nadeau, C. & Bengio, Y. <i>Inference for the Generalization Error</i> . Machine Learning 52(3), 2003. corrected CI
[6]	D'hooge, L. et al. <i>Inter-dataset generalization strength of supervised machine learning methods for intrusion detection</i> . J. Information Security and Applications 54, 2020. cross-dataset
[7]	Zaharia, M. et al. <i>Spark: Cluster computing with working sets</i> . HotCloud 2010. Spark ML
[8]	CIC & CSE. <i>CSE-CIC-IDS2018 dataset</i> . 2018. cross-dataset target

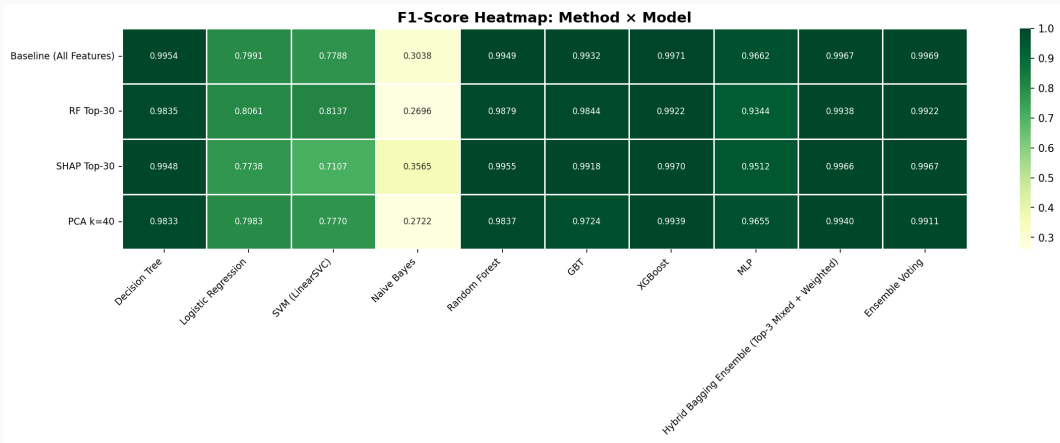
2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup slides

Backup slides

Backup: F1 heatmap, method × algorithm

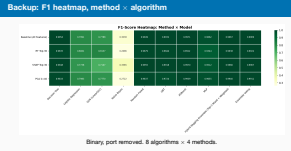


Binary, port removed, 3 algorithms × 4 methods.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: F1 heatmap, method × algorithm



Backup: hyper-parameters, fixed a priori

Model	Main hyper-parameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Shared across all reduction methods, so differences reflect the *method*, not per-model tuning.
Class weights for 6 models, scale_pos_weight for XGBoost; MLP unweighted (Spark ML has no sample weights for it) — stated in the limitations. LightGBM excluded: its SynapseML native library is x86_64-only, and the deployment target is ARM64.

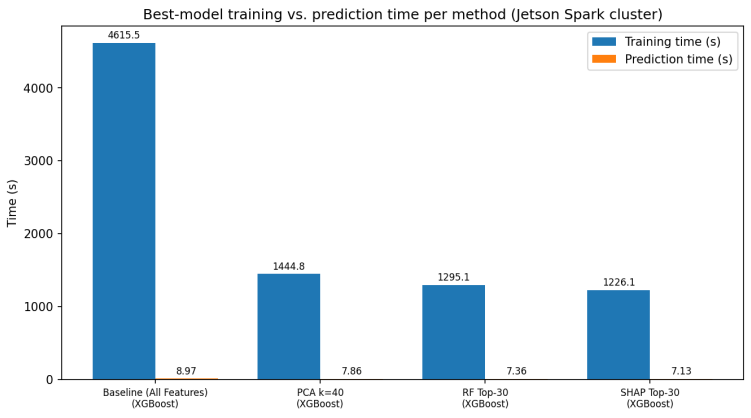
2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: hyper-parameters, fixed a priori

Backup: hyper-parameters, fixed a priori	
Model	Main hyper-parameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01
Shared across all reduction methods, so differences reflect the method, not per-model tuning. Class weights for 6 models, scale_pos_weight for XGBoost; MLP unweighted (Spark ML has no sample weights for it) — stated in the limitations. LightGBM excluded: its SynapseML native library is x86_64-only, and the deployment target is ARM64.	

Backup: training and prediction cost

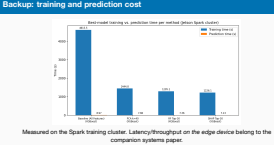


Measured on the Spark training cluster. Latency/throughput *on the edge device* belong to the companion systems paper.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: training and prediction cost



Backup: threats to validity

- **Internal.** RF/SHAP rankings come from the original training set and stay fixed across resamplings — consistent with the a-priori design, but a slight leak in the ranking step. Re-ranking per resampling is more rigorous, but costly for SHAP. MLP carries the unweighted-training caveat.
- **External.** CICIDS2017 may not reflect current traffic, and benchmark NIDS datasets have documented quality limitations. The cross-dataset conclusion rests on a *stratified sample* of CSE-CIC-IDS2018. Evasive traffic untested. Feature-level dedup itself shifts the benign/attack distribution.
- **Statistical.** $N=6$ gives low power: two-sided exact enumeration floors p at $2/2^N$, so $N=6$ just reaches 0.05. Resamplings overlap, violating t -independence — hence Nadeau–Bengio as the primary basis. Nested CV with larger N is the next reinforcement.

2026-09-26

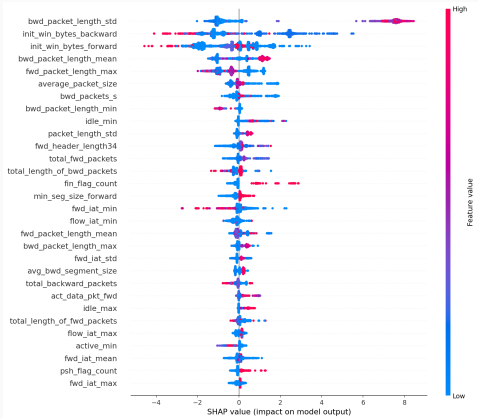
Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: threats to validity

Backup: threats to validity

- **Internal.** RF/SHAP rankings come from the original training set and stay fixed across resamplings — consistent with the a-priori design, but a slight leak in the ranking step. Re-ranking per resampling is more rigorous, but costly for SHAP. MLP carries the unweighted-training caveat.
- **External.** CICIDS2017 may not reflect current traffic, and benchmark NIDS datasets have documented quality limitations. The cross-dataset conclusion rests on a stratified sample of CSE-CIC-IDS2018. Evasive traffic untested. Feature-level dedup itself shifts the benign/attack distribution.
- **Statistical.** $N=6$ gives low power: two-sided exact enumeration floors p at $2/2^N$, so $N=6$ just reaches 0.05. Resamplings overlap, violating t -independence — hence Nadeau–Bengio as the primary basis. Nested CV with larger N is the next reinforcement.

Backup: SHAP feature attribution



SHAP is computed on a *class-stratified* sample so both benign and attack traffic are represented in the ranking.

2026-09-26

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: SHAP feature attribution

